

Codex-Memory: pamięć dla agentów Codex

1. Czym jest ten projekt

Codex-Memory, w repozytorium nazwany też Codex-Mem, to lokalny system pamięci dla agentów AI pracujących w środowisku Codex. Jego główne zadanie jest proste: agent ma nie zaczynać każdej sesji od zera.

W zwykłej pracy agent widzi bieżący prompt, fragmenty kodu i historię rozmowy. Po zakończeniu sesji wiele użytecznych informacji może jednak zniknąć z kontekstu: dlaczego wybrano daną bibliotekę, jaka komenda naprawiła testy, który błąd już raz wystąpił, jakie pliki trzeba omijać, albo jaki wzorzec w projekcie jest uznany za poprawny.

Codex-Memory dodaje warstwę trwałej wiedzy pomiędzy agentem a projektem. Ta wiedza jest zapisywana lokalnie, wyszukiwana przy kolejnych zadaniach i wstrzykiwana do kontekstu agenta wtedy, gdy pasuje do bieżącej pracy.

Najkrótszy opis działania projektu to:

```
capture -> normalize -> store -> retrieve -> rank -> inject
```

Po polsku:

- capture: system wychwytuje ważną informację z pracy agenta lub developera;
- normalize: zamienia ją w uporządkowany wpis pamięci;
- store: zapisuje wpis w lokalnej bazie;
- retrieve: wyszukuje pasujące wpisy przy kolejnym zadaniu;
- rank: wybiera najtrafniejsze wpisy;
- inject: dodaje je do kontekstu agenta.

2. Problem, który rozwiązuje

Agent Codex świetnie radzi sobie z analizą kodu, ale bez trwałej pamięci łatwo powtarza te same kroki. Może ponownie odkrywać decyzje architektoniczne, uruchamiać niepotrzebne komendy, proponować rozwiązania, które kiedyś okazały się błędne, albo pytać o rzeczy, które już były ustalone.

Dla developera oznacza to stratę czasu. Każda nowa sesja wymaga przypominania kontekstu: "tu używamy SQLite", "ten test trzeba odpalać przez uv", "tego katalogu nie zapisujemy do pamięci", "ta konfiguracja działa tylko lokalnie".

Codex-Memory zamienia takie doświadczenia w trwałą, uporządkowaną wiedzę projektu.

3. Jaką korzyść daje developerowi

Największa korzyść to mniejsza liczba powtórek. Developer nie musi za każdym razem tłumaczyć agentowi tych samych zasad projektu. Agent może sprawdzić wcześniejsze rozwiązania i szybciej dojść do poprawnej implementacji.

Praktyczne korzyści:

- szybsze wdrażanie agenta w projekt;
- mniej powtarzanych błędów;

- łatwiejsze utrzymanie decyzji technicznych;
- możliwość zapisywania sprawdzonych rozwiązań;
- lepszy kontekst przy pracy nad dużym repozytorium;
- większa spójność stylu i architektury;
- lokalna kontrola nad danymi, bo baza domyślnie działa w repozytorium;
- wygodne użycie przez CLI, API, MCP i hooki Codex.

W praktyce Codex-Memory działa jak zeszyt projektowy, z którego agent potrafi sam korzystać. Developer nadal podejmuje decyzje, ale agent ma lepszą pamięć roboczą i mniej zgaduje.

4. Główne komponenty projektu

Projekt składa się z kilku części.

apps/api

To serwis HTTP oparty o FastAPI. Jest centralnym punktem systemu. Przyjmuje nowe wpisy pamięci, zapisuje je w SQLite, obsługuje wyszukiwanie, generuje kontekst do wstrzyknięcia i wystawia diagnostykę.

Prosto mówiąc: API to "mózg operacyjny" pamięci. Inne części projektu pytają je: "zapisz to", "znajdź coś podobnego", "daj mi kontekst do tego zadania".

apps/cli

To narzędzie terminalowe napisane w TypeScript/Bun. Pozwala developerowi ręcznie zapisywać i wyszukiwać pamięć.

Przykład użycia:

```
bun run cli remember --type decision --title "Używamy SQLite lokalnie"
--context "MVP ma działać bez zewnętrznej bazy." --resolution "Vector
backend local wystarcza do developmentu."
bun run cli query "SQLite"
bun run cli debug --query "konfiguracja lokalna"
```

CLI jest wygodne, gdy developer chce jawnie zapisać ważną decyzję albo sprawdzić, co agent będzie widział.

apps/mcp-server

To serwer MCP. MCP, czyli Model Context Protocol, jest standardowym sposobem podłączania narzędzi do agentów AI. Dzięki temu agent może dostać narzędzia takie jak `store_memory`, `query_memory` i `delete_memory`.

Prosto mówiąc: MCP to przejściówka między agentem a lokalnym systemem pamięci.

plugins/codex-mem

To lokalny plugin dla Codex. Zawiera konfigurację hooków i skrypty, które uruchamiają się przy konkretnych momentach pracy agenta.

Hook to automatyczna akcja odpalana przez środowisko. W tym projekcie hook może:

- pobrać pamięć na starcie sesji;
- pobrać pamięć po wysłaniu promptu przez użytkownika;
- spróbować zapisać ważną wiedzę po zakończeniu odpowiedzi;
- wychwycić błędy z wyników narzędzi.

shared

To wspólne schematy i prompty. Schemat opisuje, jak wygląda poprawny wpis pamięci. Prompty pomagają agentowi wyciągać wiedzę i wstrzykiwać ją do kontekstu.

data

To lokalne miejsce na dane. Domyślnie baza SQLite znajduje się pod:

```
data/db/codex-mem.sqlite3
```

SQLite to lekka baza danych w jednym pliku. Nie wymaga osobnego serwera, więc dobrze pasuje do lokalnego developmentu.

5. Typy pamięci

Codex-Memory dzieli wpisy pamięci na kilka typów.

fact

Fakt, czyli obiektywna informacja o projekcie.

Przykład: "Projekt używa FastAPI jako lokalnego API pamięci."

decision

Decyzja techniczna lub architektoniczna.

Przykład: "Na start używamy SQLite zamiast Postgresa, bo projekt ma działać lokalnie bez dodatkowej infrastruktury."

bug

Opis błędu lub awarii.

Przykład: "Hook nie zapisuje pamięci, gdy API jest offline."

solution

Sprawdzone rozwiązanie problemu.

Przykład: "Aby uruchomić API, użyj `uv run --project apps/api uvicorn app.main:app --host 127.0.0.1 --port 8000`."

pattern

Wzorzec, czyli podejście warte ponownego użycia.

Przykład: "Przed zmianą zachowania pamięci najpierw sprawdź testy API i CLI."

6. Jak działa przepływ pamięci

Capture

System próbuje rozpoznać, że pojawiła się wartościowa wiedza. Może to być informacja z odpowiedzi agenta, wynik testu, błąd narzędzia, komentarz z review albo ręczny wpis developera.

Nie wszystko powinno trafić do pamięci. "OK", "gotowe" albo tymczasowy debug log to za mało. Pamięć ma być krótka, konkretna i przydatna.

Normalize

Surowa informacja jest porządkowana. System nadaje jej typ, tytuł, opis kontekstu, rozwiązanie, tagi, poziom pewności i opcjonalnie ścieżki plików.

To ważne, bo agent łatwiej wyszukuje uporządkowaną wiedzę niż luźne notatki.

Store

Wpis trafia do SQLite. API zapisuje też metadane, historię zmian i indeks pomocniczy. SQLite jest źródłem prawdy, czyli głównym miejscem, któremu system ufa.

Retrieve

Gdy developer daje agentowi nowe zadanie, system może wyszukać podobne wpisy pamięci. Wyszukiwanie bierze pod uwagę tekst zapytania, typ, tagi, projekt, ścieżki plików i daty.

Rank

Nie każdy znaleziony wpis jest równie ważny. Ranking wybiera najbardziej pasujące informacje. Dzięki temu agent nie dostaje ściany tekstu, tylko kilka najlepszych wskazówek.

Inject

Najtrafniejsze wpisy są zamieniane na krótki kontekst i dodawane do promptu agenta. Developer nie musi ich kopiować ręcznie.

7. Jak developer powinien korzystać z biblioteki

Najlepszy styl pracy to połączenie automatyki i świadomych ręcznych wpisów.

Developer powinien ręcznie zapisywać:

- decyzje architektoniczne;
- nietypowe komendy uruchomieniowe;
- rozwiązania trudnych błędów;
- ograniczenia projektu;
- zasady bezpieczeństwa;
- powtarzalne wzorce implementacyjne.

Agent powinien automatycznie korzystać z pamięci:

- przed implementacją;
- przy błędach testów;
- przy pracy w nieznanym module;
- przy decyzjach technicznych;
- przy powracających problemach.

Dobra pamięć wygląda tak:

```
Typ: solution
Tytuł: Testy API uruchamiamy przez uv
Kontekst: Projekt apps/api używa środowiska zarządzanego przez uv.
Rozwiązanie: Uruchom `uv run --project apps/api pytest apps/api/tests`.
Tagi: api, tests, uv
```

Słaba pamięć wygląda tak:

```
Tytuł: Zrobione
Kontekst: Dział.
```

Taki wpis niczego nie uczy przyszłego agenta.

8. Konfiguracja lokalna krok po kroku

Poniższe komendy zakładają Windows PowerShell i katalog główny repozytorium.

Krok 1: zainstaluj zależności JavaScript/TypeScript

```
bun install
```

Jeżeli środowisko nie ma Bun, trzeba go najpierw zainstalować. Bun uruchamia CLI oraz serwer MCP.

Krok 2: zainstaluj zależności Python API

```
uv sync --project apps/api
```

uv zarządza środowiskiem Python i zależnościami API.

Krok 3: ustaw adres API

```
$env:CODEX_MEM_API_URL = "http://127.0.0.1:8000"
```

Ten adres mówi CLI, MCP i hookom, gdzie działa lokalne API pamięci.

Krok 4: uruchom API

```
uv run --project apps/api uvicorn app.main:app --host 127.0.0.1 --port 8000
```

API powinno działać w osobnym terminalu. Można je zatrzymać przez Ctrl+C.

Krok 5: sprawdź, czy API żyje

W drugim terminalu:

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

Oczekiwana odpowiedź:

```
{ "status": "ok" }
```

Krok 6: zapisz pierwszy wpis pamięci

```
bun run cli remember --type decision --title "Lokalny backend pamięci"
--context "Development ma działać bez dodatkowych usług." --resolution
"Używamy CODEX_MEM_VECTOR_BACKEND=local oraz SQLite."
```

Krok 7: wyszukaj pamięć

```
bun run cli query "lokalny backend"
```

Krok 8: uruchom diagnostykę

```
bun run cli debug --query "konfiguracja lokalna"
```

Diagnostyka pomaga sprawdzić, czy API, baza, ranking i wstrzykiwanie kontekstu działają zgodnie z oczekiwaniami.

9. Najważniejsze zmienne środowiskowe

CODEX_MEM_API_URL

Adres API pamięci. Domyślnie:

```
http://127.0.0.1:8000
```

CODEX_MEM_DATA_DIR

Katalog danych. Domyślnie ./data.

CODEX_MEM_DB_PATH

Ścieżka do pliku SQLite. Domyślnie:

```
./data/db/codex-mem.sqlite3
```

CODEX_MEM_PROJECT

Nazwa projektu lub przestrzeni pamięci. Pozwala rozdzielać pamięć między projektami.

CODEX_MEM_INJECT_LIMIT

Ile wpisów pamięci maksymalnie wstrzyknąć do kontekstu.

CODEX_MEM_TOKEN_BUDGET

Limit rozmiaru wstrzykiwanego kontekstu. Prosto mówiąc: ile miejsca w promptcie można przeznaczyć na pamięć.

CODEX_MEM_VECTOR_BACKEND

Backend wyszukiwania podobieństwa. Domyślnie `local`. Dostępne wartości to `local`, `chroma` i `pgvector`.

Na start najlepiej użyć `local`, bo nie wymaga dodatkowych usług.

CODEX_MEM_HOOKS_ENABLED

Włącza lub wyłącza hooki Codex-Memory.

CODEX_MEM_MODE

Tryb pracy hooków:

- `active`: zapisuje i wstrzykuje pamięć automatycznie;
- `passive`: pokazuje sugestie, ale nie zapisuje automatycznie;
- `approval`: zapisuje propozycje do kolejki akceptacji;
- `debug`: pokazuje więcej informacji diagnostycznych.

10. Konfiguracja z Codex przez plugin i MCP

Projekt zawiera lokalny plugin w:

```
plugins/codex-mem
```

Plugin opisuje hooki oraz konfigurację MCP. Po włączeniu pluginu w Codex agent może automatycznie:

- pobierać pamięć na starcie sesji;
- szukać pamięci po promptcie użytkownika;
- zapisywać użyteczne informacje po zakończeniu pracy;
- reagować na błędy narzędzi.

Serwer MCP można uruchomić ręcznie:

```
$env:CODEX_MEM_API_URL = "http://127.0.0.1:8000"
bun run apps/mcp-server/src/server.ts
```

Dla narzędzi MCP ważne jest, żeby API działało wcześniej. MCP nie jest bazą danych. MCP jest mostem między agentem a API pamięci.

11. Przykładowy codzienny workflow

1. Developer uruchamia API pamięci.

2. Developer pracuje z Codexem nad zadaniem.
3. Agent pyta pamięć o podobne błędy, decyzje i wzorce.
4. Agent implementuje zmianę z lepszym kontekstem.
5. Po rozwiązaniu nietypowego problemu developer zapisuje wpis przez CLI albo pozwala hookowi go wychwycić.
6. Przy kolejnej podobnej pracy agent odzyskuje tę wiedzę automatycznie.

Przykład:

```
bun run cli remember --type solution --title "Naprawa błędu migracji SQLite"
--context "Stara baza może mieć niższy schema_version." --resolution
"Przed migracją system tworzy backup .bak obok pliku bazy."
```

Za miesiąc agent dostaje zadanie związane z migracją bazy. Zamiast odkrywać temat od nowa, może znaleźć ten wpis.

12. Proste wyjaśnienie technicznych pojęć

API

API to zestaw adresów, pod które inne programy mogą wysłać pytania lub polecenia. Tutaj API pozwala zapisywać, wyszukiwać i usuwać pamięć.

FastAPI

FastAPI to framework Pythona do budowania API. Framework to gotowy zestaw narzędzi, który pozwala szybciej tworzyć aplikacje.

SQLite

SQLite to baza danych zapisana w jednym pliku. Nie trzeba instalować osobnego serwera bazy. Dlatego jest wygodna do narzędzi lokalnych.

CLI

CLI to program używany z terminala. W tym projekcie CLI pozwala wpisywać komendy typu remember, query, get, update i debug.

MCP

MCP, czyli Model Context Protocol, to sposób, w jaki agent AI może korzystać z zewnętrznych narzędzi. Dzięki MCP Codex może dostać narzędzie query_memory i sam zapytać pamięć.

Hook

Hook to automatyczna akcja uruchamiana w konkretnym momencie. Na przykład po wysłaniu promptu albo po zakończeniu odpowiedzi agenta.

Embedding

Embedding to liczbowa reprezentacja tekstu. Dzięki niej komputer może ocenić, że dwa teksty są znaczeniowo podobne, nawet jeśli nie używają dokładnie tych samych słów.

Vector backend

Vector backend to mechanizm do wyszukiwania podobnych embeddingów. Lokalny backend działa bez dodatkowych usług. Chroma i pgvector są mocniejsze, ale wymagają osobnej infrastruktury.

Ranking

Ranking to sortowanie wyników od najbardziej do najmniej przydatnych. Dzięki rankingowi agent dostaje najlepsze wpisy, a nie wszystkie naraz.

Token budget

Token budget to limit miejsca w kontekście modelu. Model AI nie może czytać nieskończonej ilości tekstu, więc pamięć musi być krótka i dobrze dobrana.

Namespace

Namespace to oddzielna przestrzeń nazw. Pozwala rozdzielić pamięć lokalną, globalną, zespołową i współdzieloną.

Pamięć globalna

Pamięć globalna to wpisy, które mogą być użyte w wielu projektach.

Pamięć projektowa

Pamięć projektowa dotyczy konkretnego repozytorium lub konkretnego projektu.

Pamięć zespołowa

Pamięć zespołowa to lokalnie wydzielona przestrzeń dla zespołu. W obecnym stanie projektu jest to model namespace w SQLite, a nie pełna zdalna usługa z autoryzacją.

13. Bezpieczeństwo i dobre praktyki

Do pamięci nie należy zapisywać sekretów, tokenów, haseł, kluczy API, danych osobowych ani wrażliwych danych systemowych.

To bardzo ważne, bo pamięć jest tworzona po to, żeby była ponownie czytana i udostępniana agentowi. Jeśli wpiszesz sekret do pamięci, możesz później przypadkowo wstrzyknąć go do promptu.

Dobre praktyki:

- zapisuj tylko wiedzę techniczną, która może być jawna w projekcie;
- dodawaj tagi, żeby ułatwić wyszukiwanie;
- zapisuj rozwiązania dopiero po weryfikacji;
- aktualizuj albo usuwaj nieaktualne wpisy;
- używaj trybu `passive` lub `approval`, jeśli nie chcesz automatycznego zapisu.

14. Jak sprawdzić, czy wszystko działa

Minimalna ścieżka weryfikacji:

```
Invoke-RestMethod http://127.0.0.1:8000/health
bun run cli remember --type fact --title "Smoke test" --context "Sprawdzam
    lokalną pamięć."
bun run cli query "Smoke test"
bun run cli debug --query "Smoke test"
```

Jeżeli odpowiedzi wracają bez błędów, lokalna pamięć działa.

Dodatkowo można uruchomić testy:

```
bun run typecheck
bun run cli:test
bun run api:test
```

15. Najczęstsze problemy

API nie odpowiada

Sprawdź, czy terminal z uvicorn nadal działa i czy CODEX_MEM_API_URL wskazuje na `http://127.0.0.1:8000`.

CLI nie może połączyć się z API

Ustaw zmienną:

```
$env:CODEX_MEM_API_URL = "http://127.0.0.1:8000"
```

Hooki niczego nie zapisują

Sprawdź:

```
$env:CODEX_MEM_HOOKS_ENABLED  
$env:CODEX_MEM_MODE
```

W trybie `passive` hooki mogą tylko pokazywać sugestie. W trybie `approval` wpisy trafiają do kolejki, a nie bezpośrednio do bazy.

Wyszukiwanie zwraca za dużo albo za mało

Zmień limit albo profil:

```
bun run cli query "migracja sqlite" --profile short  
bun run cli query "migracja sqlite" --profile deep
```

Backend Chroma lub pgvector nie działa

Na start wróć do lokalnego backendu:

```
$env:CODEX_MEM_VECTOR_BACKEND = "local"
```

16. Podsumowanie

Codex-Memory to lokalna, uporządkowana pamięć dla agenta Codex. Projekt zapisuje sprawdzone fakty, decyzje, błędy, rozwiązania i wzorce, a potem podaje je agentowi wtedy, gdy są potrzebne.

Dla developera oznacza to mniej powtarzania, mniej utraconego kontekstu i bardziej konsekwentną pracę z agentem. Codex staje się nie tylko wykonawcą poleceń, ale partnerem, który uczy się historii projektu i potrafi ją wykorzystać w kolejnych sesjach.

Najprostszy sposób startu:

```
bun install  
uv sync --project apps/api  
$env:CODEX_MEM_API_URL = "http://127.0.0.1:8000"  
uv run --project apps/api uvicorn app.main:app --host 127.0.0.1 --port 8000
```

W drugim terminalu:

```
bun run cli remember --type fact --title "Codex-Memory działa lokalnie"  
--context "API, CLI i SQLite są skonfigurowane."  
bun run cli query "Codex-Memory"
```

Od tego momentu projekt ma pamięć, do której developer i agent mogą wracać.

Wygenerowano lokalnie z docs/codex-memory-opis-pl.md.